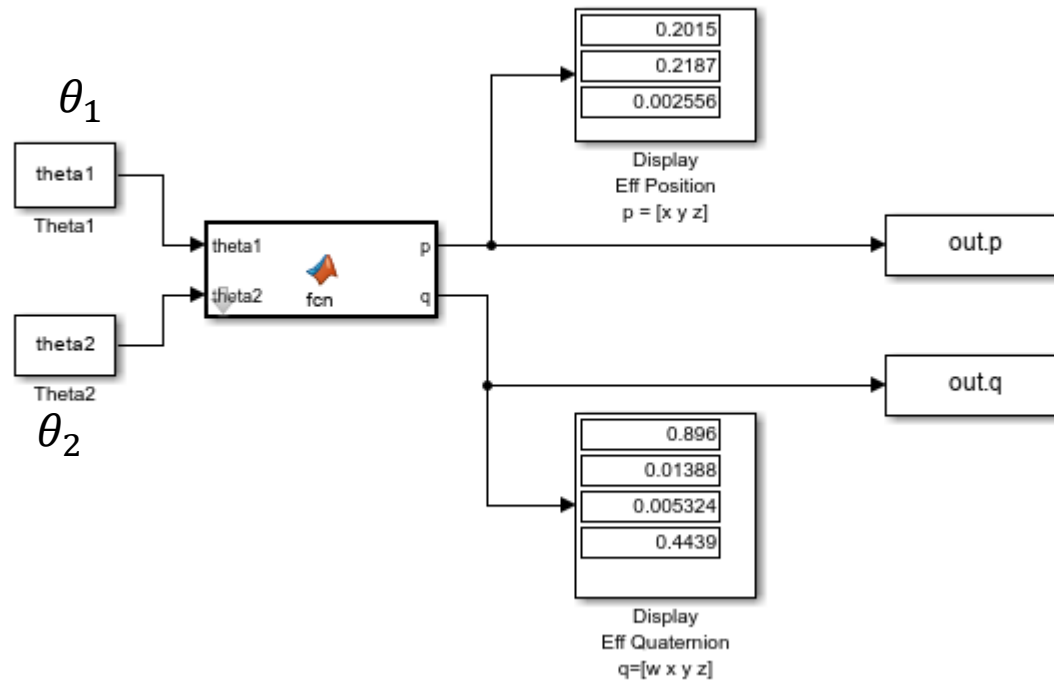
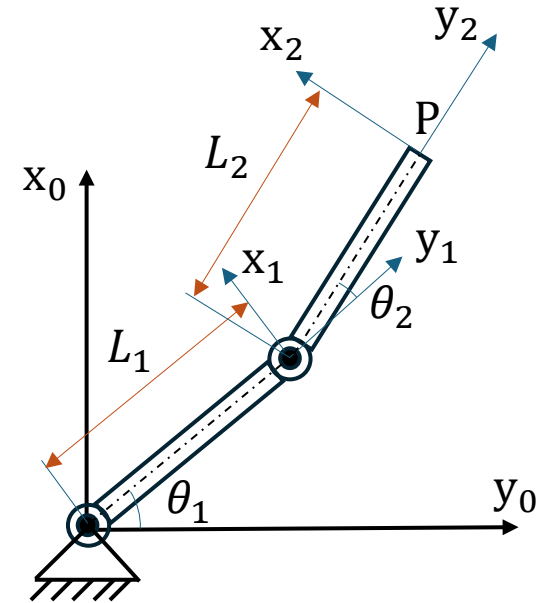


Robot calibration

2DoF planar robot

	d	θ	a	α
Link 1	0	θ_1	$L_1=0.1$ [m]	0
Link 2	0	θ_2	$L_2=0.2$ [m]	0



$[x_P \ y_P \ z_P]^T$: position of point P

$[q_x \ q_y \ q_z \ q_w]^T$: quaternion of end effector

getSimdata.m

```
clear all
% initialize random theta 1 and theta 2 values for training and testing
```

```
NTrain = 100;
```

```
NTest = 100;
```

```
theta1_train = rand(1,NTrain);
```

```
theta2_train = rand(1,NTrain);
```

Define θ_1 , θ_2 for training

```
theta1_test = rand(1,NTest);
```

```
theta2_test = rand(1,NTest);
```

Define θ_1 , θ_2 for testing

```
% send to simulink model to get the end effector results
```

```
pTrain = zeros(3,NTest); % end effector position [training]
```

```
qTrain = zeros(4,NTest); % end effector quaternion [training]
```

$pTrain = [x_P \ y_P \ z_P]^T$: position of point P for parameter identification (training)

$qTrain = [q_x \ q_y \ q_z \ q_w]^T$: quaternion of end effector (training)

```
pTest = zeros(3,NTest); % end effector position [testing]
```

```
qTest = zeros(4,NTest); % end effector quaternion [testing]
```

$pTest = [x_P \ y_P \ z_P]^T$: position of point P for testing

$qTest = [q_x \ q_y \ q_z \ q_w]^T$: quaternion of end effector (testing)

```
% training data
```

```
for i=1:NTrain
```

```
simIn = Simulink.SimulationInput("robotModel_blackbox"); %create object
```

```
theta1 = theta1_train(i);
```

```
theta2 = theta2_train(i);
```

```
out = sim(simIn); %run simulation, all results returned in "out"
```

```
% end effector position
```

```
pTrain(:,i) = out.p.data(:,end);
```

```
% end effector quaternion
```

```
qTrain(:,i) = out.q.data(:,end);
```

```
end
```

To get training data

Input the Simulink model

Insert the θ_1 , θ_2 values

Run simulation

Get position of P and quaternion of end effector

```
% testing data
```

```
for i=1:NTest
```

```
simIn = Simulink.SimulationInput("robotModel_blackbox"); %create object
```

Same for testing data

```
theta1 = theta1_test(i);
```

```
theta2 = theta2_test(i);
```

```
out = sim(simIn); %run simulation, all results returned in "out"
```

```
% end effector position
```

```
pTest(:,i) = out.p.data(:,end);
```

```
% end effector quaternion
```

```
qTest(:,i) = out.q.data(:,end);
```

```
end
```

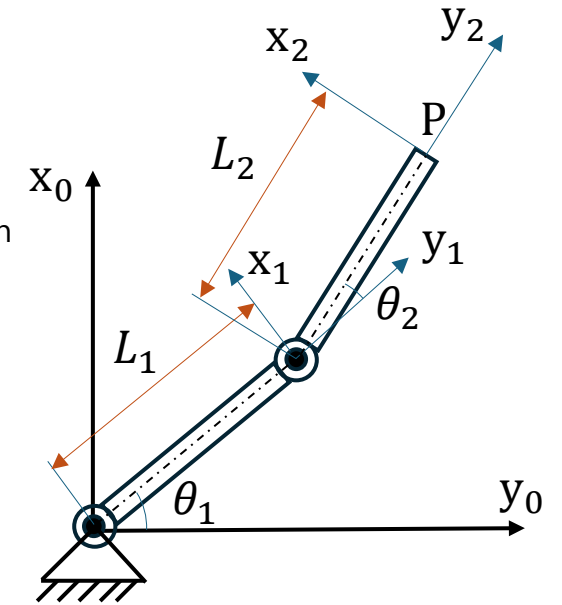


Fig. 1. 2dof planar robot

Linearized optimization algorithm

$\mathbf{x} = \mathbf{x}_0$

While err > Err do:

$$\mathbf{P}_{\text{est}} = \mathbf{P}(\mathbf{q}, \mathbf{x}) + \mathbf{J}\Delta\mathbf{x} \text{ (Linearize model)}$$

$$\Delta\mathbf{x} = \arg \min_{\Delta\mathbf{x}} \frac{1}{2} \|\mathbf{P}_{\text{real}} - \mathbf{P}_{\text{est}}\|^2 \rightarrow \Delta\mathbf{x} = \mathbf{J}^+(\mathbf{P}_{\text{real}} - \mathbf{P}_{\text{est}})$$

$$\mathbf{x} = \mathbf{x} + \Delta\mathbf{x}$$

end while

Improve algorithm:

$\mathbf{x} = \mathbf{x}_0$

While err > Err do:

$$\Delta\mathbf{x} = \arg \min_{\Delta\mathbf{x}} \frac{1}{2} \|\mathbf{W}(\mathbf{P}_{\text{real}} - \mathbf{P}(\mathbf{q}, \mathbf{x})) - \mathbf{W}\mathbf{J}\mathbf{W}_p\Delta\mathbf{x}\|^2 + \lambda\|\Delta\mathbf{x}\|^2$$

$$\rightarrow \Delta\mathbf{x} = \mathbf{H}^+ \mathbf{g}, \text{ where } \mathbf{H} = \mathbf{W}_p^T \mathbf{J}^T \mathbf{W}^T \mathbf{W} \mathbf{J} \mathbf{W}_p \text{ and } \mathbf{g} = \mathbf{W}_p^T \mathbf{J}^T \mathbf{W}^T \mathbf{W} (\mathbf{P}_{\text{real}} - \mathbf{P}(\mathbf{q}, \mathbf{x}))$$

$$\mathbf{x} = \mathbf{x} + \Delta\mathbf{x}$$

end while

$\mathbf{W}$: to assign the weights for each data point or choose which Cartesian component to optimize for
 $\mathbf{W}$ has size (num_dim X m), where num_dim is dimension of Cartesian coordinates to optimize for, m is the number of data points.

$\mathbf{W}_p$: to chose which parameters of the DH matrix to optimize for
 $\mathbf{W}_p$ has size (4n x 4n), where n is the number of joints

λ : to limit the linearized model and actual model
(chosen following Levenberg-Marquardt algorithm)

How to run:

- 1) run getSimdata.m to get data set (theta1, theta2, P, quaternion) (training and testing) using blackbox model